

Sprawozdanie końcowe

Aplikacja webowa do zarządzania zadaniami

Bartłomiej Karkoszka

Rafał Grot

Filip Stępień

Przedmiot: Modelowanie i Analiza Systemów Informatycznych

Grupa: 1ID21A

Spis treści

1	Opis tematu projektu	3
2	Skład zespołu	3
3	Założenia projektowe	3
3.1	Założenia funkcjonalne	3
3.2	Założenia нефункционалне	3
3.3	Założenia technologiczne	4
4	Architektura i diagramy	4
4.1	Diagram przypadków użycia	4
4.2	Diagram klas (model dziedziny)	5
4.3	Diagram komponentów	6
4.4	Diagram warstw	8
4.5	Diagram sekwencji – utworzenie zadania	8
4.6	Diagram aktywności	9
5	Graf zależności pomiędzy modułami	10
5.1	Zależności w backendzie (Go)	10
5.2	Zależności we frontendzie (React + Redux)	11
5.3	Komunikacja między aplikacjami	12
6	Metryki kodu	12
6.1	Liczba plików i linii kodu (tokei)	12
6.2	Cyklomatyczna złożoność (gocyclo)	13
6.3	Frontend – struktura	13

7	Stan implementacji	14
7.1	Struktura repozytorium	14
7.2	Lista endpointów REST	14
7.3	Schemat bazy danych	15
8	Kluczowe fragmenty kodu	15
8.1	Middleware uwierzytelniania (Go)	15
8.2	Warstwa serwisów – tworzenie zadania (Go)	16
8.3	Walidacja DTO (Go)	16
8.4	Slice Redux z EntityAdapter (TypeScript)	17
8.5	Asynchroniczny thunk (TypeScript)	18
8.6	Inicjalizacja Keycloak po stronie klienta (TypeScript)	18
9	Instrukcja uruchomienia	19
9.1	Wymagania wstępne	19
9.2	Skrypt uruchomieniowy	20
9.3	Pierwsze logowanie	20
9.4	Dostępne usługi – podsumowanie	20
10	Wnioski końcowe	21

1 Opis tematu projektu

Tematem projektu jest **webowa aplikacja do zarządzania zadaniami** (typu *TODO list*) przeznaczona dla użytkowników indywidualnych. System pozwala zalogowanemu użytkownikowi tworzyć, edytować, oznaczać statusem oraz usuwać własne zadania, a także przeglądać podsumowanie statystyczne w formie panelu *dashboard*. Każde zadanie posiada tytuł, opcjonalny opis, status realizacji, poziom priorytetu oraz opcjonalny termin wykonania.

Projekt został zrealizowany jako pełnoprawna aplikacja typu *full-stack*: część kliencka (SPA w React 19) komunikuje się poprzez REST API z częścią serwerową napisaną w języku Go. Funkcje uwierzytelniania i autoryzacji zostały delegowane do zewnętrznego serwisu **Keycloak** (protokoły OAuth2 / OpenID Connect), co eliminuje konieczność przechowywania haseł w bazie aplikacji. Cały system jest dostarczony jako zestaw kontenerów Docker, dzięki czemu jego uruchomienie wymaga jedynie posiadania środowiska **docker compose**.

2 Skład zespołu

- **Bartłomiej Karkoszka** – modelowanie, dokumentacja, frontend,
- **Rafał Grot** – backend (Go), baza danych, integracja z Keycloak, frontend,
- **Filip Stępień** – frontend (React, Redux Toolkit), warstwa prezentacji.

Wszyscy członkowie zespołu uczestniczyli również w przygotowaniu dokumentacji projektowej oraz w przeglądach kodu (*code review*) realizowanych poprzez *pull requesty* w repozytorium Git.

3 Założenia projektowe

3.1 Założenia funkcjonalne

- Rejestracja, logowanie i wylogowanie użytkownika realizowane przez serwis Keycloak.
- Tworzenie zadania z tytułem, opisem, priorytetem i terminem realizacji.
- Edycja i usuwanie własnych zadań użytkownika.
- Zmiana statusu zadania (Do zrobienia / W trakcie / Zrobione).
- Filtrowanie listy zadań po statusie oraz priorytecie.
- Panel *dashboard* z podsumowaniem liczby zadań w poszczególnych statusach.

3.2 Założenia нефunkcjonalne

- Responsywny interfejs użytkownika (desktop, tablet, mobile).

- Bezpieczeństwo: hasła użytkowników **nie są** przechowywane w bazie aplikacji – uwierzytelnianie poprzez Keycloak, autoryzacja oparta o tokeny JWT weryfikowane przez JWKS.
- Łatwe wdrożenie poprzez **docker compose** – jedna komenda uruchamia bazę danych, Keycloak oraz Adminera.
- Wersjonowanie kodu w Git (repozytorium monorepo).
- Statyczne typowanie zarówno po stronie backendu (Go) jak i frontendu (TypeScript), generowane zapytania SQL (**sqlc**) oraz generowany klient REST (**Orval**) zapewniają spójność typów między warstwami.

3.3 Założenia technologiczne

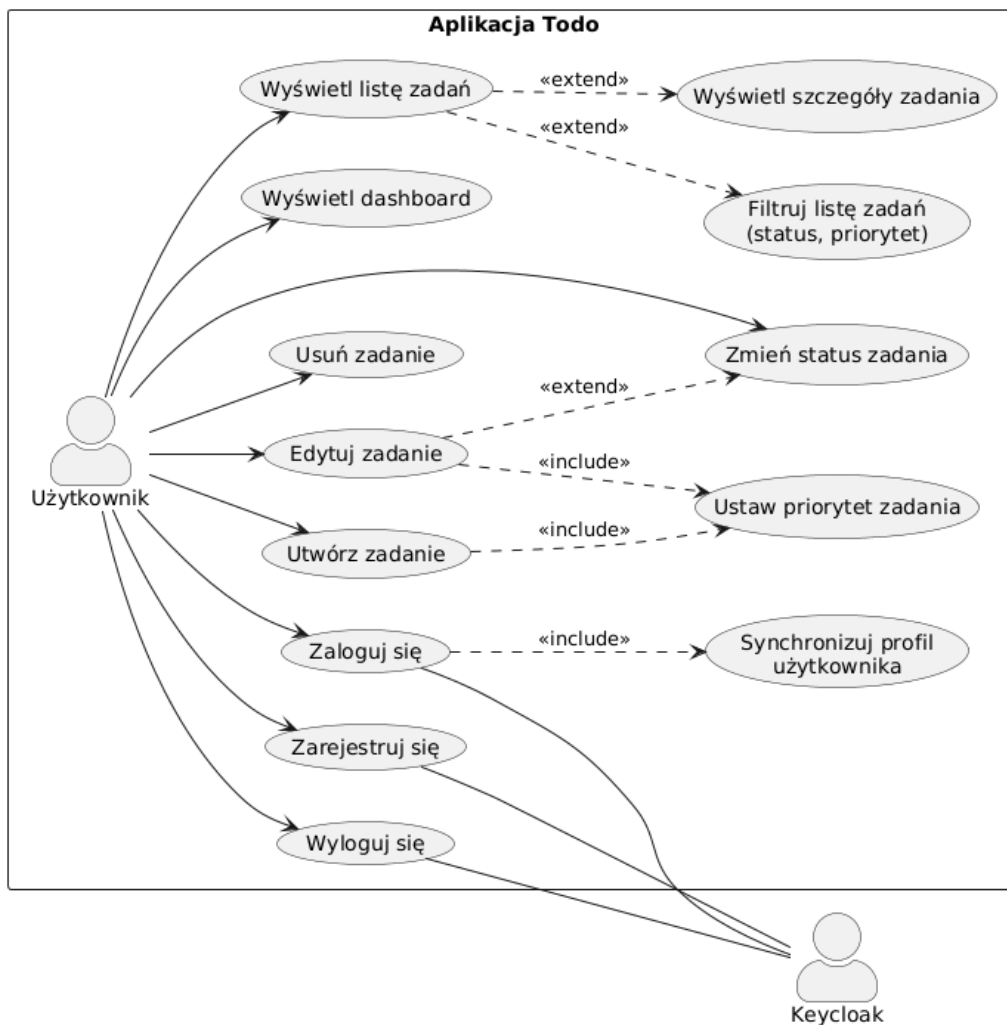
- **Frontend:** React 19, TypeScript, Vite, Tailwind CSS v4, shadcn/ui, React Router v7, Redux Toolkit (z `createEntityAdapter`).
- **Backend:** Go (router **chi**, generator zapytań **sqlc**, migracje **goose**, logger **slog**, dokumentacja **swaggo**).
- **Baza danych:** PostgreSQL 16.
- **Uwierzytelnianie:** Keycloak 26.2 (OAuth2 / OIDC, PKCE, JWKS).
- **Konteneryzacja:** Docker, Docker Compose.

W stosunku do kamienia milowego I jedyną istotną zmianą jest zastąpienie zaplanowanego Spring Boot (Java) przez Go po stronie backendu – decyzja podyktowana chęcią wypróbowania języka w praktyce.

4 Architektura i diagramy

4.1 Diagram przypadków użycia

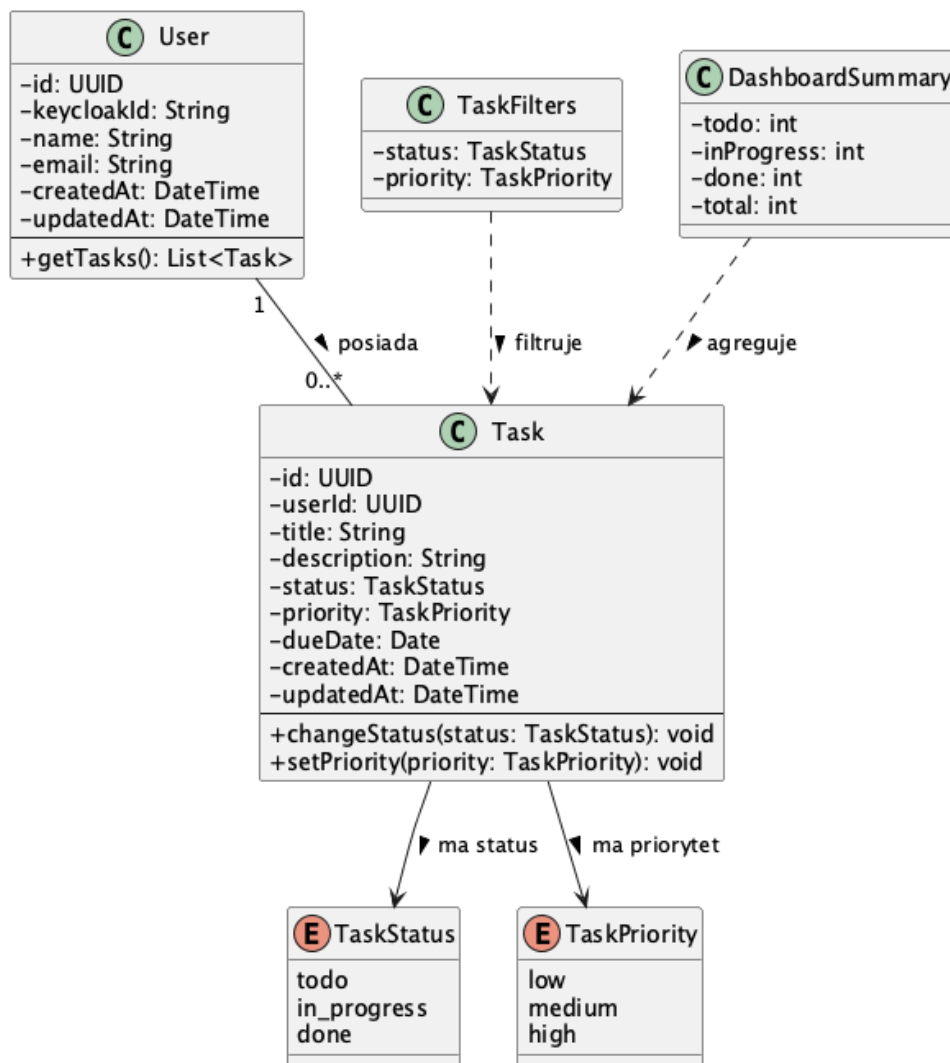
Diagram przedstawia interakcje aktora *Użytkownik* (umieszczonego wewnątrz granicy systemu) z aplikacją oraz powiązanie z zewnętrznym dostawcą tożsamości (Keycloak). Przypadek *Synchronizuj profil użytkownika* (relacja `<<include>>`) odpowiada rzeczywistemu mechanizmowi tworzenia rekordu w tabeli **users** przy pierwszym żądaniu autoryzowanym.



Rysunek 1: Diagram przypadków użycia

4.2 Diagram klas (model dziedziny)

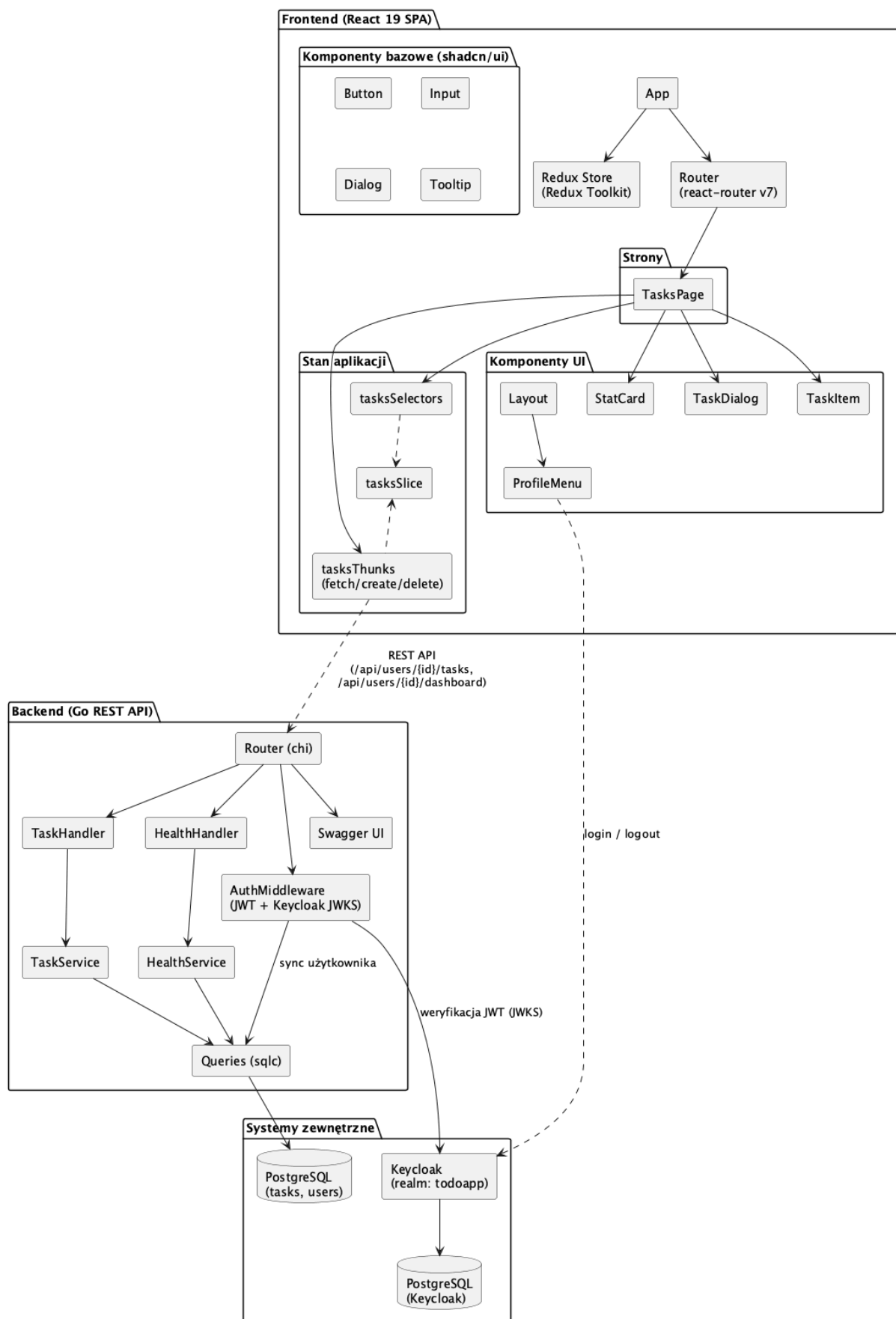
Model dziedziny zawiera dwie główne encje: `User` oraz `Task`, powiązane relacją *1 do 0..**. Typy wyliczeniowe `TaskStatus` (`todo`, `in_progress`, `done`) oraz `TaskPriority` (`low`, `medium`, `high`) zostały zaimplementowane jako natywne typy enum w PostgreSQL. Klasy pomocnicze `TaskFilters` oraz `DashboardSummary` odpowiadają DTO obecnym w kodzie.



Rysunek 2: Diagram klas

4.3 Diagram komponentów

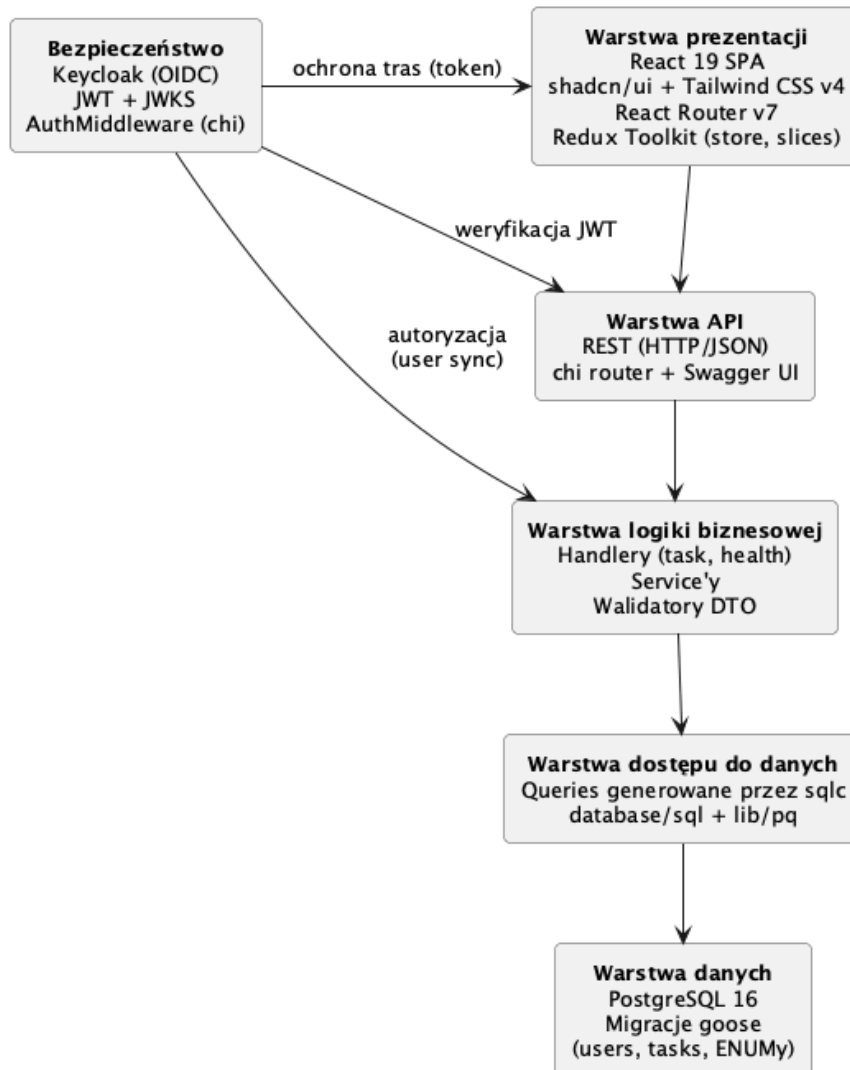
Diagram odzwierciedla rzeczywiste pakiety: po stronie frontendu wydzielony jest *Redux Store* (slice, thunki, selektory) oraz komponenty UI (*Layout*, *TaskDialog*, *TaskItem*, *StatCard*, *ProfileMenu*). Po stronie backendu przedstawiono router *chi*, *AuthMiddleware* z weryfikacją JWKS, handlery, warstwę serwisów oraz wygenerowane przez *sqlc* Queries.



Rysunek 3: Diagram komponentów

4.4 Diagram warstw

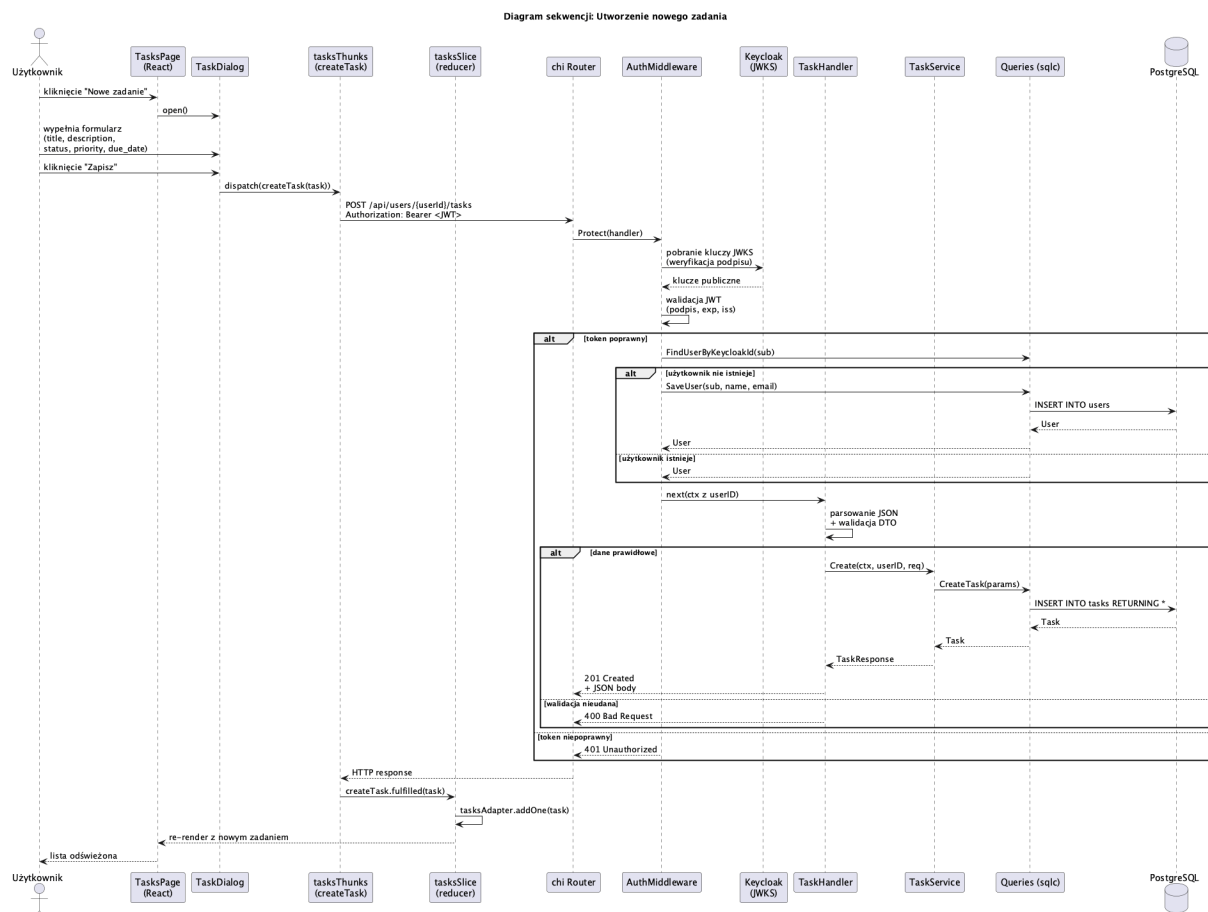
Architektura wpisuje się w klasyczny model warstwowy: prezentacja (React + Redux), API (chi, Swagger UI), logika (handlers, serwisy), dostęp do danych (sqlc), dane (PostgreSQL) oraz przekrojowa warstwa bezpieczeństwa (Keycloak, JWT).



Rysunek 4: Diagram warstw systemu

4.5 Diagram sekwencji – utworzenie zadania

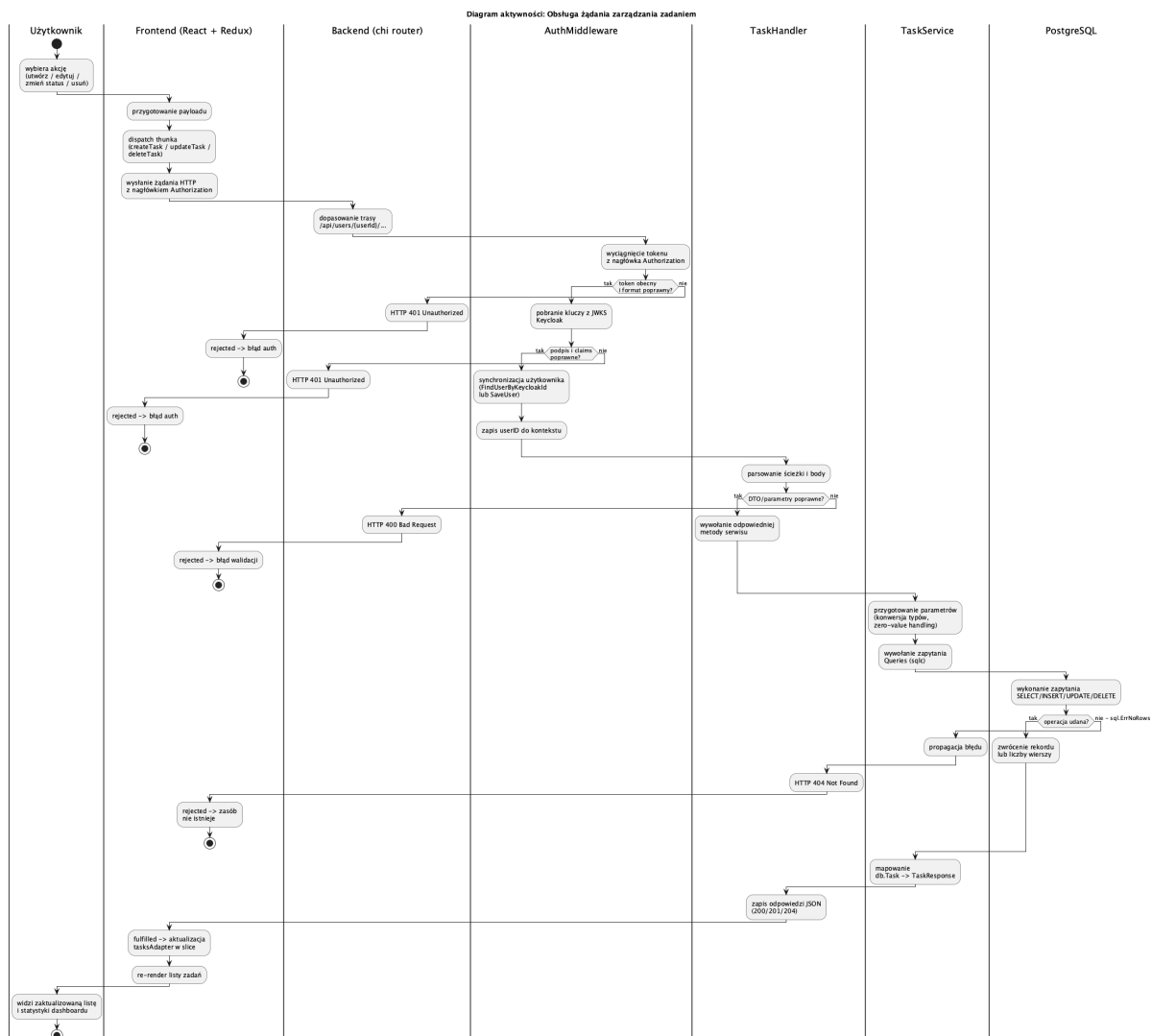
Diagram ilustruje pełny przepływ przez wszystkie warstwy systemu: od interakcji w UI, przez akcję Redux i żądanie HTTP, weryfikację tokenu, synchronizację użytkownika, walidację DTO, persystencję w bazie, aż po aktualizację stanu po stronie klienta.



Rysunek 5: Diagram sekwencji – utworzenie nowego zadania

4.6 Diagram aktywności

Diagram aktywności pokazuje ogólną obsługę żądania zarządzania zadaniem wraz z punktami decyzyjnymi dla istotnych ścieżek błędów obecnych w implementacji: 401 (brak / niepoprawny token), 400 (błędne dane wejściowe) oraz 404 (`sql.ErrNoRows`).



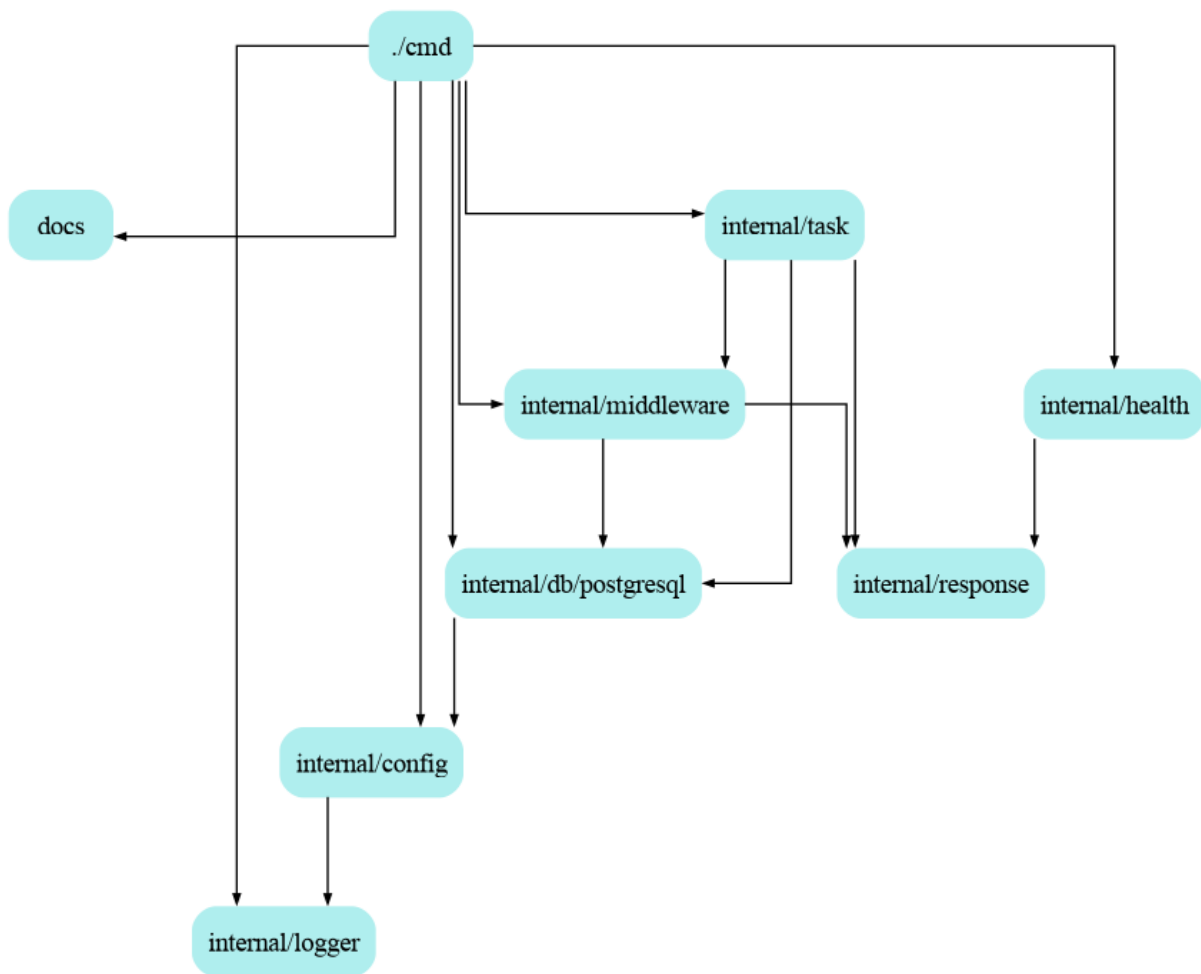
Rysunek 6: Diagram aktywności – obsługa żądania zarządzania zadaniem

5 Graf zależności pomiędzy modułami

System zorganizowany jest jako *monorepo* z dwoma aplikacjami: `apps/task-api` (backend Go) oraz `apps/task-frontend` (frontend React). Przedstawione poniżej grafy zostały wygenerowane automatycznie z kodu źródłowego: dla backendu narzędziem `godepgraph` (analiza importów pakietów Go) renderowanym przez Graphviz, dla frontendu narzędziem `madge` (analiza importów plików TypeScript / TSX).

5.1 Zależności w backendzie (Go)

Graf zależności pomiędzy pakietami w obrębie modułu `task-api` (bez bibliotek zewnętrznych oraz standardowej biblioteki Go).

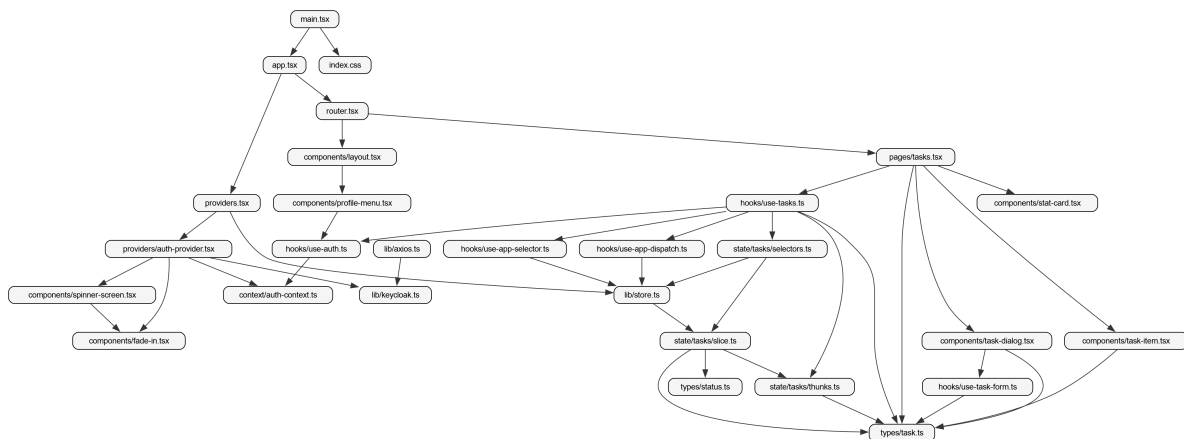


Rysunek 7: Graf zależności pakietów backendu (godepgraph + Graphviz)

Punkt wejścia (`cmd`) zależy bezpośrednio od wszystkich pakietów aplikacyjnych. Pakiet `internal/response` jest wspólną zależnością warstwy HTTP (`middleware`, `task`, `health`). Warstwa danych (`internal/db/postgresql`) jest wykorzystywana zarówno przez `middleware` (synchronizacja profilu użytkownika), jak i przez `task` (CRUD zadań).

5.2 Zależności we frontendzie (React + Redux)

Graf zależności modułów frontendu zaczyna się od `main.tsx` i rozgałęzia się na trzy poddrzewa: routing (`router.tsx` – `pages/`), warstwę uwierzytelniania (`providers/auth-provider.tsx`) oraz store Redux (`lib/store.ts` – `state/tasks/`). W grafie pominięto komponenty bazowej biblioteki `shadcn/ui` (`components/ui/`) oraz pomocniczy plik `lib/utils.ts` celem zachowania czytelności rysunku.



Rysunek 8: Graf zależności modułów frontendu (madge + Graphviz)

5.3 Komunikacja między aplikacjami

- **Frontend – Backend:** komunikacja po REST/JSON pod endpointami `/api/users/{id}/tasks` oraz `/api/users/{id}/dashboard`. Klient HTTP (axios) automatycznie dołącza nagłówek `Authorization: Bearer <JWT>`.
- **Frontend – Keycloak:** przepływ OAuth2 / OIDC w wariantcie *authorization code* z PKCE, realizowany przez bibliotekę `keycloak-js`.
- **Backend – Keycloak:** backend pobiera klucze publiczne z JWKS pod ścieżką `/realms/todoapp/protocol/openid-connect/certs` i weryfikuje podpis tokenów JWT przy pomocy biblioteki `MicahParks/keyfunc`.
- **Backend – PostgreSQL:** pula połączeń `database/sql`, zapytania generowane przez `sqlc`.

6 Metryki kodu

Metryki zostały zebrane przy pomocy narzędzi `tokei` (liczba plików i linii kodu) oraz `gocyclo` (cyklomatyczna złożoność funkcji w języku Go).

6.1 Liczba plików i linii kodu (tokei)

Language	Files	Lines	Code	Comments	Blanks
CSS	1	130	125	0	5
Go	19	1501	1194	95	212
HTML	1	13	13	0	0
JavaScript	1	26	25	0	1
JSON	5	140	136	0	4
SQL	4	80	49	16	15
SVG	2	25	25	0	0
TSX	29	2235	2039	10	186
TypeScript	19	350	299	1	50
YAML	2	97	81	0	16

Markdown	2	128	0	94	34
- BASH	1	5	5	0	0
(Total)		133	5	94	34
=====					
Total	85	4730	3991	216	523
=====					

6.2 Cyklomatyczna złożoność (gocyclo)

Średnia cyklomatyczna złożoność funkcji w kodzie Go wynosi **2,55**. Zdecydowana większość funkcji ma złożoność 1–3 (wartości uznawane za *trywialne* w skali McCabe’a). Poniżej surowy wynik `gocyclo -top 15` (kolumny: złożoność, pakiet, funkcja, plik z numerem linii):

```
10 task (*service).GetAll apps/task-api/internal/task/service.go:29:1
6 task (*service).GetDashboard apps/task-api/internal/task/service.go:74:1
6 task (*Handler).Update apps/task-api/internal/task/handler.go:164:1
6 postgresql (*Queries).FindAllTasksByUserIdAndStatus
↳ apps/task-api/internal/db/postgresql/tasks.sql.go:186:1
6 postgresql (*Queries).FindAllTasksByUserIdAndPriority
↳ apps/task-api/internal/db/postgresql/tasks.sql.go:144:1
6 postgresql (*Queries).FindAllTasksByUserId
↳ apps/task-api/internal/db/postgresql/tasks.sql.go:102:1
6 postgresql (*Queries).CountTasksByUserIdGroupedByStatus
↳ apps/task-api/internal/db/postgresql/tasks.sql.go:24:1
5 task (*service).Update apps/task-api/internal/task/service.go:130:1
5 task (*service).Create apps/task-api/internal/task/service.go:103:1
5 task (*Handler).Create apps/task-api/internal/task/handler.go:125:1
5 task (TaskFilters).Validate apps/task-api/internal/task/dto.go:104:1
5 middleware (*AuthMiddleware).syncUser
↳ apps/task-api/internal/middleware/auth.go:77:1
4 task (*Handler).Delete apps/task-api/internal/task/handler.go:206:1
4 task (*Handler).GetByID apps/task-api/internal/task/handler.go:95:1
4 task (*Handler).GetAll apps/task-api/internal/task/handler.go:34:1
```

6.3 Frontend – struktura

Po stronie React zaimplementowano:

- **1 stronę** (`TasksPage`),
- **7 komponentów aplikacyjnych** (`Layout`, `TaskDialog`, `TaskItem`, `ProfileMenu`, `StatCard`, `FadeIn`, `SpinnerScreen`),
- **13 komponentów UI** z biblioteki `shadcn/ui`,
- **6 customowych hooków** (`useTasks`, `useTaskForm`, `useAuth`, `useAppDispatch`, `useAppSelector`, `useMobile`),
- **1 slice Redux** z `createEntityAdapter` oraz **4 asynchroniczne thunki** (`fetchTasks`, `createTask`, `updateTask`, `deleteTask`).

7 Stan implementacji

7.1 Struktura repozytorium

```
todoapp/
|-- apps/
|   |-- task-api/           # backend Go (REST API)
|   |   |-- cmd/           # main.go, api.go - bootstrap aplikacji
|   |   |-- internal/
|   |       |-- config/     # parametry srodowiskowe
|   |       |-- db/postgresql # modele, queries (sqlc), migracje
|   |       |-- health/     # endpoint /health
|   |       |-- logger/     # slog wrapper
|   |       |-- middleware/ # AuthMiddleware (JWT, JWKS)
|   |       |-- response/   # pomocnicze pisanie JSON
|   |       |-- task/       # handler, service, dto
|   |-- task-frontend/     # frontend React (SPA)
|   |-- src/
|       |-- components/    # Layout, TaskDialog, TaskItem, ...
|       |-- pages/         # TasksPage
|       |-- hooks/         # custom hooks
|       |-- state/tasks/   # slice, thunks, selectors
|       |-- providers/     # AuthProvider
|       |-- lib/           # store, axios, keycloak, env
|       |-- generated/     # Orval - klient REST
|-- docker/keycloak/realm.json
|-- docker-compose.yml
|-- flake.nix
`-- Taskfile.yml
```

Struktura repozytorium

7.2 Lista endpointów REST

Wszystkie endpointy są zabezpieczone middleware `Protect` wymagającym nagłówka `Authorization: Bearer <JWT>` i udostępnione pod prefiksem `/api` (konfigurowalnym przez zmienną `API_BASE_PATH`).

Metoda	Ścieżka	Opis	Kody
GET	<code>/health</code>	Stan aplikacji i bazy danych	200, 401, 500
GET	<code>/users/{userId}/tasks</code>	Lista zadań użytkownika z filtrami	200, 400, 401, 500
POST	<code>/users/{userId}/tasks</code>	Utworzenie nowego zadania	201, 400, 401, 500
GET	<code>/users/{userId}/tasks/{id}</code>	Szczegóły pojedynczego zadania	200, 400, 401, 404, 500
PUT	<code>/users/{userId}/tasks/{id}</code>	Aktualizacja zadania	200, 400, 401, 404, 500
DELETE	<code>/users/{userId}/tasks/{id}</code>	Usunięcie zadania	204, 400, 401, 404, 500
GET	<code>/users/{userId}/dashboard</code>	Statystyki zadań użytkownika	200, 400, 401, 500

7.3 Schemat bazy danych

Migracje (goose) tworzą tabele `users` i `tasks` oraz typy wyliczeniowe `task_status` (`todo`, `in_progress`, `done`) i `task_priority` (`low`, `medium`, `high`). Tabela `tasks` posiada klucz obcy `user_id` z kaskadowym usuwaniem oraz znaczniki czasowe `created_at` i `updated_at`.

8 Kluczowe fragmenty kodu

W tej sekcji prezentujemy najistotniejsze fragmenty implementacji.

8.1 Middleware uwierzytelniania (Go)

Środkowy element warstwy bezpieczeństwa: pobiera klucze publiczne z Keycloak (JWKS), weryfikuje podpis JWT, a następnie synchronizuje profil użytkownika z lokalną bazą (rekord tworzony jest *lazy* przy pierwszym żądaniu autoryzowanym).

```
1 func (m *AuthMiddleware) Protect(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
3         tokenStr, ok := extractAuthToken(r)
4         if !ok {
5             response.Error(w, http.StatusUnauthorized, "Unauthorized")
6             return
7         }
8
9         claims, ok := m.parseTokenClaims(tokenStr)
10        if !ok {
11            response.Error(w, http.StatusUnauthorized, "Unauthorized")
12            return
13        }
14
15        uc := extractClaims(claims)
16
17        user, err := m.syncUser(r.Context(), uc)
18        if err != nil {
19            response.Error(w, http.StatusInternalServerError, "Failed to sync user")
20            return
21        }
22
23        ctx := context.WithValue(r.Context(), UserIDKey, user.ID)
24        next.ServeHTTP(w, r.WithContext(ctx))
25    })
26 }
```

internal/middleware/auth.go – weryfikacja JWT i synchronizacja użytkownika

Funkcja `syncUser` implementuje wzorzec *lazy provisioning*: jeżeli użytkownik o podanym `sub` (z tokenu JWT) nie istnieje jeszcze w tabeli `users`, zostaje utworzony nowy rekord; w przeciwnym przypadku zwracany jest istniejący.

```
1 func (m *AuthMiddleware) syncUser(ctx context.Context, uc userClaims) (db.User,
2     ↪ error) {
3     user, err := m.queries.FindUserByKeycloakId(ctx, uc.sub)
4     if err != nil && !errors.Is(err, sql.ErrNoRows) {
5         return db.User{}, err
6     }
7 }
```

```

7   if errors.Is(err, sql.ErrNoRows) {
8       user, err = m.queries.SaveUser(ctx, db.SaveUserParams{
9           KeycloakID: uc.sub,
10          Name:      uc.name,
11          Email:     uc.email,
12      })
13      if err != nil {
14          return db.User{}, err
15      }
16  }
17
18  return user, nil
19 }

```

internal/middleware/auth.go – syncUser

8.2 Warstwa serwisów – tworzenie zadania (Go)

Serwis odpowiada za parsowanie daty, mapowanie pól opcjonalnych na typy `sql.NullString` / `sql.NullTime` oraz wywołanie wygenerowanego przez `sqlc` zapytania.

```

1  func (s *service) Create(ctx context.Context, userID uuid.UUID, req
   ↪ CreateTaskRequest) (TaskResponse, error) {
2      params := db.CreateTaskParams{
3          UserID:  userID,
4          Title:   req.Title,
5          Status:  db.TaskStatus(req.Status),
6          Priority: db.TaskPriority(req.Priority),
7      }
8
9      if req.Description != nil {
10         params.Description = sql.NullString{String: *req.Description, Valid: true}
11     }
12
13     if req.DueDate != nil {
14         t, err := time.Parse(time.DateOnly, *req.DueDate)
15         if err != nil {
16             return TaskResponse{}, err
17         }
18         params.DueDate = sql.NullTime{Time: t, Valid: true}
19     }
20
21     task, err := s.queries.CreateTask(ctx, params)
22     if err != nil {
23         return TaskResponse{}, err
24     }
25     return toTaskResponse(task), nil
26 }

```

internal/task/service.go – Create

8.3 Walidacja DTO (Go)

Walidacja wykonywana jest na poziomie handlera, przed wejściem do warstwy serwisów. Każde DTO eksponuje metodę `Validate()`, która kompozycyjnie wykorzystuje wspólne funkcje pomocnicze (`validateStatus`, `validatePriority`, `validateDueDate`).


```

1 var (
2     validStatuses = map[string]struct{}{"todo": {}, "in_progress": {}, "done": {}}
3     validPriorities = map[string]struct{}{"low": {}, "medium": {}, "high": {}}
4 )
5
6 func validateStatus(s string) error {
7     if _, ok := validStatuses[s]; !ok {
8         return errors.New("status must be one of: todo, in_progress, done")
9     }
10    return nil
11 }
12
13 func validateDueDate(d *string) error {
14     if d == nil || *d == "" {
15         return nil
16     }
17     if _, err := time.Parse(time.DateOnly, *d); err != nil {
18         return errors.New("due_date must be in YYYY-MM-DD format")
19     }
20    return nil
21 }
22
23 func (r CreateTaskRequest) Validate() error {
24     if r.Title == "" {
25         return errors.New("title is required")
26     }
27     if err := validateStatus(r.Status); err != nil {
28         return err
29     }
30     if err := validatePriority(r.Priority); err != nil {
31         return err
32     }
33     return validateDueDate(r.DueDate)
34 }

```

internal/task/dto.go – walidacja

8.4 Slice Redux z EntityAdapter (TypeScript)

Stan zadań po stronie klienta jest znormalizowany przy pomocy `createEntityAdapter`, co umożliwia wydajne operacje *upsert* / *removeOne* bez konieczności ręcznego mapowania list. Aktualizacja flagi status (*idle* / *loading* / *error*) realizowana jest jednym matcherem dla wszystkich 4 thunków – dzięki `isAnyOf` eliminujemy duplikację kodu.

```

1 export const tasksAdapter = createEntityAdapter<Task>();
2
3 const initialState = tasksAdapter.getInitialState<{ status: FetchStatus }>({
4     status: 'idle'
5 });
6
7 const thunks = [fetchTasks, createTask, updateTask, deleteTask];
8
9 export const tasksSlice = createSlice({
10     name: 'tasks',
11     initialState,

```

```

12   reducers: {},
13   extraReducers: builder => {
14     builder
15       .addCase(fetchTasks.fulfilled, (state, action) => {
16         tasksAdapter.setAll(state, action.payload);
17       })
18       .addCase(createTask.fulfilled, (state, action) => {
19         tasksAdapter.addOne(state, action.payload);
20       })
21       .addCase(updateTask.fulfilled, (state, action) => {
22         tasksAdapter.upsertOne(state, action.payload);
23       })
24       .addCase(deleteTask.fulfilled, (state, action) => {
25         tasksAdapter.removeOne(state, action.payload);
26       })
27       .addMatcher(isAnyOf(...thunks.map(t => t.pending)), state => {
28         state.status = 'loading';
29       })
30       .addMatcher(isAnyOf(...thunks.map(t => t.fulfilled)), state => {
31         state.status = 'idle';
32       })
33       .addMatcher(isAnyOf(...thunks.map(t => t.rejected)), state => {
34         state.status = 'error';
35       });
36   }
37 });

```

src/state/tasks/slice.ts

8.5 Asynchroniczny thunk (TypeScript)

Thunki opakowują wywołania REST generowane przez `Orval`. Cykl życia (`pending` / `fulfilled` / `rejected`) jest automatycznie odzwierciedlany w stanie store przy pomocy matcherów ze `slice'a`.

```

1  export const createTask = createAsyncThunk<Task, { userId: string } & Task>(
2    'tasks/create',
3    async ({ userId, ...task }) => {
4      const req: TaskCreateTaskRequest = {
5        title: task.title,
6        description: task.description,
7        status: task.status,
8        priority: task.priority,
9        due_date: task.dueDate || undefined
10     };
11
12     const data = await postUsersUserIdTasks(userId, req);
13     return toTask(data);
14   }
15 );

```

src/state/tasks/thunks.ts – createTask

8.6 Inicjalizacja Keycloak po stronie klienta (TypeScript)

Provider montowany jest na samym szczycie drzewa komponentów – bez prawidłowego tokenu cała aplikacja jest niedostępna (`onLoad: 'login-required'`). Token jest automatycznie od-

świeżany 30 sekund przed wygaśnięciem; w przypadku niepowodzenia użytkownik zostaje wylogowany. Strażnik `keycloak.didInitialize` chroni przed podwójną inicjalizacją w trybie *React StrictMode*.

```
1 export function AuthProvider({ children }: { children: ReactNode }) {
2   const [isLoading, setIsLoading] = useState(true);
3
4   useEffect(() => {
5     if (keycloak.didInitialize) {
6       return;
7     }
8
9     keycloak.onTokenExpired = () => {
10       keycloak.updateToken(30).catch(() => keycloak.logout());
11     };
12
13     keycloak.init({ onLoad: 'login-required' }).then(() => setIsLoading(false));
14   }, []);
15
16   if (isLoading) {
17     return <SpinnerScreen />;
18   }
19
20   const claims = ClaimsSchema.parse(keycloak.tokenParsed);
21
22   const value: AuthContextValue = {
23     user: {
24       id: claims.sub,
25       name: claims.name,
26       email: claims.email
27     },
28     logout: () => keycloak.logout()
29   };
30
31   return (
32     <AuthContext.Provider value={value}>
33       <FadeIn>{children}</FadeIn>
34     </AuthContext.Provider>
35   );
36 }
```

src/providers/auth-provider.tsx

9 Instrukcja uruchomienia

9.1 Wymagania wstępne

- Docker oraz Docker Compose,
- Go w wersji 1.22 lub wyższej,
- Node.js w wersji 20 lub wyższej (`npm`),
- opcjonalnie: `task` (Taskfile) – dla skrótów,
- opcjonalnie: Nix flake (`flake.nix`) – dla powtarzalnego środowiska.

9.2 Skrypt uruchomieniowy

Poniższy skrypt zawiera komplet kroków potrzebnych do uruchomienia systemu na świeżym środowisku.

```
# Clone the repository and enter the project root
git clone <repository>
cd todoapp

# Copy the environment file (.env holds DB credentials,
# Keycloak URLs, etc.; defaults work for local setup)
cp .env.example .env

# Start infrastructure containers:
# PostgreSQL (5432), Keycloak's PostgreSQL (5433),
# Keycloak (8080), Adminer (8081)
docker compose up -d

# Run database migrations (creates users, tasks tables and enums).
# Taskfile loads .env automatically via the `dotenv` directive.
cd apps/task-api
task migrate:up

# Start the Go REST API (`task run` loads .env and runs `go run ./cmd`).
# REST API: http://localhost:3000/api
# Swagger UI: http://localhost:3000/swagger/index.html
task run
cd ../../..

# Start the React SPA - served on http://localhost:5173
cd apps/task-frontend
npm install
npm run dev
```

Pełny skrypt uruchomieniowy

9.3 Pierwsze logowanie

Po otwarciu <http://localhost:5173> aplikacja automatycznie przekieruje na ekran logowania Keycloak. Można zalogować się jako użytkownik domyślny (user@example.com / `user`) albo założyć nowe konto poprzez link *Register*.

9.4 Dostępne usługi – podsumowanie

Usługa	URL	Domyślne dane
Frontend (SPA)	http://localhost:5173	–
REST API	http://localhost:3000/api	wymagany JWT
Swagger UI	http://localhost:3000/swagger/	OAuth2 (swagger-ui)
Keycloak (panel admin)	http://localhost:8080/admin	admin / admin
Adminer (baza)	http://localhost:8081	todoapp / todoapp

Tabela 2: Adresy usług po pełnym uruchomieniu systemu

10 Wnioski końcowe

Realizacja celów. Wszystkie założenia funkcjonalne sformułowane w kamieniu milowym I zostały zaimplementowane. System pozwala na rejestrację i logowanie poprzez Keycloak, pełne zarządzanie zadaniami (CRUD), filtrowanie po statusie i priorytecie oraz prezentację statystyk w formie panelu *dashboard*.

Jakość kodu. Metryki potwierdzają wysoką jakość kodu: niska złożoność cykliczna, brak długich i skomplikowanych funkcji oraz jasny podział odpowiedzialności pomiędzy warstwami. Po stronie backendu żądanie HTTP trafia kolejno do handlera, serwisu i wreszcie do warstwy zapytań wygenerowanych przez `sqlc`. Po stronie frontendu strona korzysta z hooków, hooki dispatchują akcje na store Redux, a store komunikuje się z API poprzez thunki i wygenerowanego klienta. Cała komunikacja między warstwami przebiega przez interfejsy, co ułatwia testowanie jednostkowe.

Bezpieczeństwo. Zastosowanie zewnętrznego serwera Keycloak (OAuth2 / OIDC) wraz z weryfikacją podpisu tokenów JWT po stronie backendu (JWKS) zapewnia solidną podstawę bezpieczeństwa: hasła użytkowników nigdy nie trafiają do aplikacji, a rotacja kluczy realizowana jest po stronie Keycloak. Mechanizm *lazy provisioning* użytkowników (`syncUser`) sprawia, że żaden rekord nie powstaje bez wcześniej zweryfikowanej tożsamości.

Podsumowanie. W ramach projektu dostarczono działający, dwuwarstwowy system webowy spełniający wszystkie postawione wymagania funkcjonalne i нефункционалне. Stos technologiczny (Go + React + Keycloak + PostgreSQL) okazał się trafnym wyborem dla aplikacji o tej skali, a zastosowanie generatorów kodu (`sqlc`, `Orval`, `swaggo`) pozwoliło utrzymać spójność dokumentacji z kodem na każdym etapie pracy.